



Riddle Game

Submitted By : Lakshya Varshney

Registration ID : 25BAI10822

Abstract

Riddle Game is a simple console-based Java application in which the player is shown randomly selected riddles and enters an answer. The program checks the answer without considering letter case, increases the score for a correct response, and provides an optional hint when the answer is incorrect. The player can continue playing or exit at any time. The project demonstrates fundamental Java programming concepts in an interactive and easy-to-understand form.

Introduction

The purpose of this project is to build a small interactive game using core Java. It is designed for beginners who want practical experience with input/output, arrays, string comparison, random selection, loops, and conditional statements. Because the application runs in the console, it does not require any external libraries or graphical interface.



Objective

- Create an interactive riddle-based game using Java.
- Store multiple riddles, answers, and hints using arrays.
- Select a riddle randomly for each round.
- Accept and validate user input.
- Maintain and display the player's score.
- Demonstrate loops, conditions, strings, and Scanner input.

Technology And Requirement

Item	Requirement
Programming Language	Java
JDK	JDK 8 or later
Interface	Command Line / Console
Input	Keyboard via Scanner
External Libraries	None

Working of the Project

When the program starts, the score is initialized to zero. Three arrays store the riddles, their corresponding answers, and hints. Inside a loop, `Math.random()` is used to select a random riddle. The selected question is printed and the player enters an answer. The answer is checked using `equalsIgnoreCase()`, so uppercase and lowercase differences do not affect the result. For a correct answer, the score increases by one. For an incorrect answer, the player is asked whether a hint is required. Finally, the program asks whether the player wants another round. If the response is not 'yes', the loop ends and the final score is displayed.

Main Java Concepts Use

Arrays: Three String arrays store riddles, answers, and hints.

Scanner: Reads answers and menu responses from the keyboard.

while loop: Repeats the game until the player chooses to stop.

if-else: Determines whether the answer is correct and whether a hint should be shown.

Math.random(): Generates a random index for selecting a riddle.

String methods: equalsIgnoreCase() compares answers without case sensitivity, while toLowerCase() simplifies yes/no input handling.

Algorithm

1. Start the program.
2. Create a Scanner object and set score = 0.
3. Initialize arrays for riddles, answers, and hints.
4. Generate a random index.
5. Display the selected riddle.
6. Read the player's answer.
7. Compare the answer with the correct answer.
8. If correct, increase score; otherwise offer an optional hint.
9. Ask whether the player wants to continue.
10. Repeat from step 4 if the answer is yes.
11. Display the final score and end the program.

Expected Outcome

A typical run displays a randomly selected riddle, accepts the player's response, reports whether it is correct, optionally displays a hint, and then asks whether the player wants to continue. At the end, it prints the total score and a closing message.

Example:

Riddle: The more you take, the more you leave behind. What am I?

User: Footsteps

Correct!

Continue playing? (yes/no)

User: no

Your final score: 1

Thanks for playing!

Advantages

- Simple and beginner-friendly.
- No external dependencies.
- Demonstrates several important Java fundamentals.
- Random question selection makes repeated play less predictable.
- Easy to extend with additional riddles.

Limitations

- **The game has only three riddles in the current version.**
 - **Answers are stored directly in the source code.**
 - **There is no graphical user interface.**
- **The same riddle can appear repeatedly because selection is random without a used-question tracker.**
 - **The score is not saved after the program closes.**

Conclusion

The Riddle Game successfully demonstrates how basic Java programming concepts can be combined to create an interactive application. The project uses arrays for data storage, random number generation for question selection, Scanner for input, string comparison for answer checking, conditions for decision-making, and a loop for repeated gameplay. It provides a strong beginner-level foundation for developing larger Java applications.

Thank You